

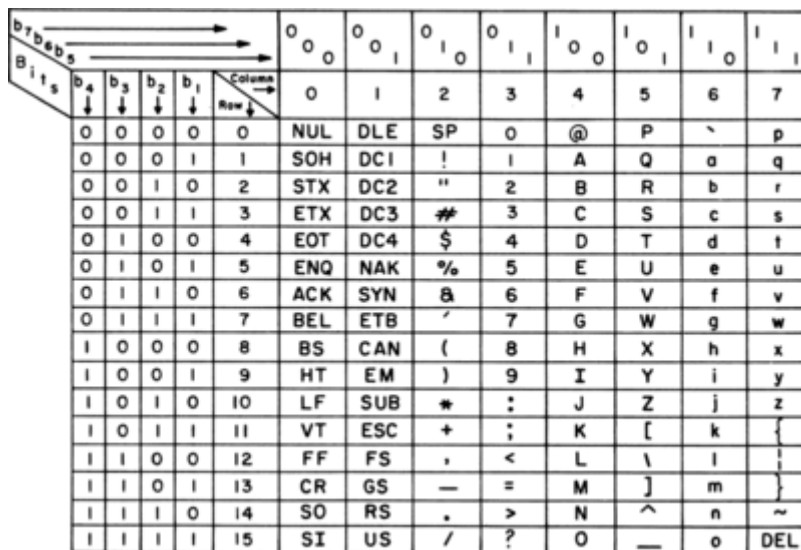
Rappresentazione dei caratteri

Codifica ASCII standard

Il problema

Dopo aver visto numeri naturali, interi e reali, come possiamo rappresentare dei caratteri con dei bit? Semplice, associamo ad ogni carattere un numero naturale! Si crea così una tabella che associa ad ogni numero da 0 a n (da definire) un determinato carattere. Ora: se contiamo solo le lettere maiuscole e minuscole, sono 52, quindi $\lceil \log_2(52) \rceil = 6$, ci servirebbero 6 bit per rappresentarle. Dobbiamo però tenere in considerazione anche le cifre da 0 a 9, qualche carattere utile (segni di interpunzione, parentesi varie, operatori matematici) e qualche carattere di controllo per le periferiche; 6 bit non bastano! Nel lontano 1961 Bob Bemer, ingegnere dell'IBM, stilò una lista dei caratteri strettamente necessari e propose una codifica a 7 bit, che prese il nome di ASCII (American Standard Code for Information Interchange).

Codifica e decodifica



b7 b6 b5 b4 b3 b2 b1							Column Row							
b7 b6 b5 b4 b3 b2 b1							0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	NUL	DLE	SP	0	@	P	^	p
0	0	0	0	1	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	0	1	0	2	2	STX	DC2	"	2	B	R	b	r
0	0	0	1	1	3	3	ETX	DC3	#	3	C	S	c	s
0	0	1	0	0	4	4	EOT	DC4	\$	4	D	T	d	t
0	0	1	0	1	5	5	ENQ	NAK	%	5	E	U	e	u
0	0	1	1	0	6	6	ACK	SYN	&	6	F	V	f	v
0	0	1	1	1	7	7	BEL	ETB	'	7	G	W	g	w
0	1	0	0	0	8	8	BS	CAN	(	8	H	X	h	x
0	1	0	0	1	9	9	HT	EM	)	9	I	Y	i	y
0	1	0	0	1	10	10	LF	SUB	*	:	J	Z	j	z
0	1	0	1	1	11	11	VT	ESC	+	;	K	[	k	{
0	1	1	0	0	12	12	FF	FS	,	<	L	\	l	
0	1	1	0	1	13	13	CR	GS	-	=	M	]	m	}
0	1	1	1	0	14	14	SO	RS	.	>	N	^	n	~
0	1	1	1	1	15	15	SI	US	/	?	O	_	o	DEL

<
https://upload.wikimedia.org/wikipedia/commons/thumb/c/cf/USASCII_code_chart.png/400px-USASCII_code_chart.png>

Tabella originale di codifica ASCII.

Dec	Hex	Oct	Bin	Char	Dec	Hex	Oct	Bin	Char	Dec	Hex	Oct	Bin	Char	Dec	Hex	Oct	Bin	Char
0	0x00	000	0000000	NUL	32	0x20	040	0100000	space	64	0x40	100	1000000	@	96	0x60	140	1100000	`
1	0x01	001	0000001	SOH	33	0x21	041	0100001	!	65	0x41	101	1000001	A	97	0x61	141	1100001	a
2	0x02	002	0000010	STX	34	0x22	042	0100010	"	66	0x42	102	1000010	B	98	0x62	142	1100010	b
3	0x03	003	0000011	ETX	35	0x23	043	0100011	#	67	0x43	103	1000011	C	99	0x63	143	1100011	c
4	0x04	004	0000100	EOT	36	0x24	044	0100100	\$	68	0x44	104	1000100	D	100	0x64	144	1100100	d
5	0x05	005	0000101	ENQ	37	0x25	045	0100101	%	69	0x45	105	1000101	E	101	0x65	145	1100101	e
6	0x06	006	0000110	ACK	38	0x26	046	0100110	&	70	0x46	106	1000110	F	102	0x66	146	1100110	f
7	0x07	007	0000111	BEL	39	0x27	047	0100111	'	71	0x47	107	1000111	G	103	0x67	147	1100111	g
8	0x08	010	0001000	BS	40	0x28	050	0101000	(	72	0x48	110	1001000	H	104	0x68	150	1101000	h
9	0x09	011	0001001	TAB	41	0x29	051	0101001	)	73	0x49	111	1001001	I	105	0x69	151	1101001	i
10	0x0A	012	0001010	LF	42	0x2A	052	0101010	*	74	0x4A	112	1001010	J	106	0x6A	152	1101010	j
11	0x0B	013	0001011	VT	43	0x2B	053	0101011	+	75	0x4B	113	1001011	K	107	0x6B	153	1101011	k
12	0x0C	014	0001100	FF	44	0x2C	054	0101100	,	76	0x4C	114	1001100	L	108	0x6C	154	1101100	l
13	0x0D	015	0001101	CR	45	0x2D	055	0101101	-	77	0x4D	115	1001101	M	109	0x6D	155	1101101	m
14	0x0E	016	0001110	SO	46	0x2E	056	0101110	.	78	0x4E	116	1001110	N	110	0x6E	156	1101110	n
15	0x0F	017	0001111	SI	47	0x2F	057	0101111	/	79	0x4F	117	1001111	O	111	0x6F	157	1101111	o
16	0x10	020	0010000	DLE	48	0x30	060	0110000	0	80	0x50	120	1010000	P	112	0x70	160	1110000	p
17	0x11	021	0010001	DC1	49	0x31	061	0110001	1	81	0x51	121	1010001	Q	113	0x71	161	1110001	q
18	0x12	022	0010010	DC2	50	0x32	062	0110010	2	82	0x52	122	1010010	R	114	0x72	162	1110010	r
19	0x13	023	0010011	DC3	51	0x33	063	0110011	3	83	0x53	123	1010011	S	115	0x73	163	1110011	s
20	0x14	024	0010100	DC4	52	0x34	064	0110100	4	84	0x54	124	1010100	T	116	0x74	164	1110100	t
21	0x15	025	0010101	NAK	53	0x35	065	0110101	5	85	0x55	125	1010101	U	117	0x75	165	1110101	u
22	0x16	026	0010110	SYN	54	0x36	066	0110110	6	86	0x56	126	1010110	V	118	0x76	166	1110110	v
23	0x17	027	0010111	ETB	55	0x37	067	0110111	7	87	0x57	127	1010111	W	119	0x77	167	1110111	w
24	0x18	030	0011000	CAN	56	0x38	070	0111000	8	88	0x58	130	1011000	X	120	0x78	170	1111000	x
25	0x19	031	0011001	EM	57	0x39	071	0111001	9	89	0x59	131	1011001	Y	121	0x79	171	1111001	y
26	0x1A	032	0011010	SUB	58	0x3A	072	0111010	:	90	0x5A	132	1011010	Z	122	0x7A	172	1111010	z
27	0x1B	033	0011011	ESC	59	0x3B	073	0111011	;	91	0x5B	133	1011011	[	123	0x7B	173	1111011	{
28	0x1C	034	0011100	FS	60	0x3C	074	0111100	<	92	0x5C	134	1011100	\	124	0x7C	174	1111100	
29	0x1D	035	0011101	GS	61	0x3D	075	0111101	=	93	0x5D	135	1011101	]	125	0x7D	175	1111101	}
30	0x1E	036	0011110	RS	62	0x3E	076	0111110	>	94	0x5E	136	1011110	^	126	0x7E	176	1111110	~
31	0x1F	037	0011111	US	63	0x3F	077	0111111	?	95	0x5F	137	1011111	_	127	0x7F	177	1111111	DEL

<https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/ascii-cheat-sheet.png?ssl=1>

Tabella ASCII semplificata. Sarà questa quella disponibile all'esame, salvo diversa indicazione!

Sopra potete vedere due tabelle per codificare e decodificare i caratteri: se l'uso della seconda è immediato (**sarà quella disponibile all'esame!**), la prima ha bisogno di un paio di chiarimenti. La riga in alto rappresenta i primi 3 bit della codifica; subito sotto ci sono le corrispondenti cifre in esadecimale. Le prime 4 colonne sono i successivi 4 bit (potete vedere $b_3, b_4 \dots$) e vicino le relative codifiche esadecimali. Quindi, dopo aver individuato il carattere da codificare, si prendono i relativi primi tre bit (determinati dalla colonna in cui si trova) e i restanti 4 (determinati dalla riga). Per esempio, per il carattere "T", vediamo che sta nella colonna 101 e nella riga 0100, quindi la sua codifica sarà 1010100. Per la codifica esadecimale basta prendere i numeri esadecimali, in questo caso 5 e 4 (0x54). Per decodificare, si prendono i primi tre bit e si identifica la colonna, e con i restanti la riga. La casella selezionata ci darà il carattere! Potremo anche trovare tabelle dove righe e colonne sono invertite; l'importante è ricordare che prima va il gruppo da 3 bit, poi quello da 4. Un paio di osservazioni: le cifre sono ordinate in ordine crescente, idem le lettere maiuscole e poco dopo le minuscole (così, data una lettera minuscola, la maiuscola relativa si trova 32 "posti" prima e viceversa).

N.B.: la maggior parte di noi pronuncia "ASCII" come "asci" (come "ascia" ma senza "a" finale), ma la pronuncia inglese originale è "aschi" (come "husky" ma senza pronunciare la "h"!). Per i fanatici dell'IPA: non è /'affi/, ma /'æski/!

Esempio 1

Codifichiamo la parola “Aereo” in binario, codifica ASCII a 7 bit. Prendiamo la codifica delle singole lettere:

A: 1000001

e: 1100101

r: 1110010

e: 1100101

o: 1101111

Aereo: 1000001 1100101 1110010 1100101 1101111

Et voilà!

Esempio 2

Decodifichiamo questo carattere ASCII: 1011101

...basta scorrere la tabella finché non lo troviamo! Possiamo semplificarci la vita convertendolo in esadecimale (raggruppandolo 4 bit alla volta partendo da destra e convertendo i numeri così ottenuti):

101 1101

7 13

7 D

0x7D

Probabilmente così è più semplice cercare e trovare che è la parentesi quadra chiusa, “]”!

L'ottavo bit e il parity check

Visto che normalmente “impacchettiamo” le informazioni a gruppi di 8 bit, ossia un byte, ci troveremmo con un bit in più. Se è necessario avere la codifica su un byte anziché su 7 bit, il suddetto bit (LSB) ridondante viene usato per il *parity check*: è impostato a 1 se c'è un numero dispari di 1, 0 altrimenti. Aggiungendo quel bit, si avrà sempre che il numero di bit uguali a 1 è pari (se nella codifica sono dispari, il bit di parità è 1, quindi globalmente sono pari; se sono già pari nella codifica, il bit di parità non aggiunge alcun 1 e restano quindi pari). Se al momento

della decodifica si hanno un numero dispari di 1, significa che qualcosa è andato storto nella trasmissione dei dati. Se non specificato, comunque, convertiranno normalmente a 7 bit!

Codifica ASCII estesa

Il bit in più per raggiungere il byte, però, può anche essere usato per rappresentare altri caratteri! Infatti, se con 7 bit ne abbiamo 128, con 8 raddoppiamo a 256. Nasce così la codifica ASCII estesa! I caratteri in più variano da paese a paese in relazione alle necessità; per esempio, noi italiani abbiamo bisogno delle lettere accentate, gli spagnoli dell'n con la tilde (ñ), i danesi della o barrata ("aptang", ø), la lista è immensa. Ecco le principali estensioni (non bisogna saperle a memoria!):

- ISO-8859-1(Latin-1), utilizzato nella Zona Europea Occidentale
- ISO-8859-2 (Latin-2), utilizzato zona Europea Orientale (Serbia, Albania, Ungheria, Romania)
- ISO-8859-3 (Latin-3), utilizzato nell' Europea del Sud (Malta), include l'Esperanto
- ISO-8859-4 (Latin-4), obsoleto
- ISO-8859-5 (Part 5, Cyrillic), alfabeto Cirillico
- ISO-8859-6 (Part 6, Arabic), alfabeto Arabo
- ISO-8859-7 (Part 7, Greek), alfabeto Greco
- ISO-8859-8 (Part 8, Hebrew), alfabeto Ebraico

Anche se l'ASCII è accettato da tutti i computer, nasce il problema di avere una codifica anche per migliaia di altri caratteri (alfabeto coreano, caratteri cinesi, giapponesi e così via). Non possiamo certo creare centinaia di estensioni! La soluzione arriverà con UNICODE.

Codifica UNICODE

Arriviamo così ad una codifica su 16 bit (che verrà poi estesa a 24 e 32) che permette di rappresentare tutti i caratteri nelle principali lingue del mondo. Questo è UNICODE, il sistema di codifica indipendente dalla lingua, dal sistema operativo e dal programma utilizzato! Un carattere UNICODE è identificato da un numero esadecimale a 8 cifre (quindi 4 byte, o 32 bit), detto *code point*; viene solitamente scritto dopo U+, che è come 0x per gli esadecimali. Avendo 32 bit possiamo rappresentare con la stessa codifica caratteri di alfabeti diversi, rendendo possibili documenti multilingue altrimenti impossibili con l'ASCII esteso, ma anche la codifica di emoji o altri simboli utili (per esempio il doppio integrale, ∫∫ (U+222C): non ditemi che non è utile! 🤪 (anche quest'emoji è in UNICODE, U+1F61C)) .

Standard di codifica UTF-8

Il bello di avere 32 bit a disposizione è che si possono rappresentare più di 4 miliardi di caratteri, ma rispetto ad una codifica a 8 bit occupa il quadruplo dello spazio. Vengono così definite varie codifiche più compatte, che permettono appunto di risparmiare memoria. La più usata è l'UTF-8 (Unicode Transformation Format a 8 bit), che lascia invariata la codifica a 7 bit ASCII per i primi 128 caratteri, aggiungendo 0 come MSB, e i successivi li divide in gruppi di 2, 3 o 4 byte in base alla necessità. Se servono tra 8 e 11 bit per un carattere, per esempio, li si distribuisce su 2 byte: il primo comincerà con 110 (indica che ne servono 2) e il secondo con 10 (indica che il byte fa parte di un carattere a più byte). Visto che l'MSB è 1, automaticamente ci troviamo di fronte ad un carattere non-ASCII! Vediamo più nello specifico:

N. byte	Struttura	Bit utili
1 byte	0xxxxxxx	7
2 byte	110xxxxx 10xxxxxx	11
3 byte	1110xxxx 10xxxxxx 10xxxxxx	16
4 byte	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	21

Lo standard UTF-8 viene usato per codificare le stringhe nei programmi Java, fra le altre cose! Questo significa che Java riesce a gestire UNICODE nativamente.

E anche per oggi è tutto! Non ci sono esempi per le ultime due codifiche perché non abbiamo mai fatto esercizi, e servirebbero comunque le rispettive tabelle nel caso: se ci diranno che una certa tabella sarà disponibile all'esame, allora inizieremo a preoccuparcene 😊